Banking Management System

Open Ended Lab Project

Submitted By:
Nida Hameed – 2024-SE-41
Hassan Gillani – 2024-SE-42
Daniyal Ayub – 2024-SE-20

September 27, 2025

Contents

1 Introduction

The Banking Management System is a console-based application developed in **C++** to simulate core banking operations. Designed as an open-ended lab assignment, it showcases **object-oriented programming**, **data structures**, and **financial transaction handling**. The system supports multi-currency transactions, **account management**, **loan processing**, and secure authentication with passwords.

This project provides a practical foundation for understanding banking operations and can be extended for advanced features.

2 Project Overview

The system enables users to **create accounts**, perform deposits, withdrawals, transfers, **apply for loans**, **pay installments**, and view transaction histories. All transactions are managed in **Pakistani Rupees (PKR)** as the base currency, with support for **USD** and **EUR** conversions.

Key Technologies Used:

Technology	Description
C++ Standard Library	iostream, string, vector, map, etc.
Enumerations	For categories and currencies
Classes	Account and Bank
Structs	Transaction and Loan

Table 1: Technologies Used in the Project

3 Features

- **Account Creation**: Supports savings, current, or fixed deposit accounts in PKR, USD, or EUR.
- **Login and Authentication**: Secure login with account number and password.
- **Deposits and Withdrawals**: Multi-currency with insufficient funds validation.
- **Transfers**: Between accounts in the system.
- **Transaction History**: Logs dates, types, amounts, and details.
- **Loan Management**: Apply, pay, and view loans with principal, interest, and term.
- **Currency Conversion**: Uses fixed exchange rates (e.g., 1 USD = 280 PKR).

- **Balance Inquiry**: Displays balance in chosen currency.
- **Input Validation**: Ensures valid numeric inputs.

4 System Design

4.1 Entities and Relationships

- **Currencies**: Enum for PKR, USD, EUR with conversion via exchange rate map.
- **Accounts**: Class managing balance (in PKR), transactions, and loans.
- **Transactions**: Struct for date, category, amount, currency, details.
- **Loans**: Struct for principal, interest, term, and payment methods.
- **Bank**: Class managing account vector and operations.

4.2 Data Flow

1. User interacts with **Bank** class menu.
2. Post-login, access **Account** methods.
3. Operations update PKR balance and log transactions.
4. Loans are credited and paid via balance deductions.

Assumptions:

- Static exchange rates.
- Simple interest calculation.
- In-memory data, lost on exit.

5 Code Structure and Explanation

5.1 Header Files Used

5.2 Classes and Objects

- **Account Class**: Manages customer accounts with private members and methods.
- **Bank Class**: Oversees all accounts with a single instance.

Header	Purpose
iostream	Input/output operations
string	String handling
vector	Dynamic arrays for transactions/loans
ctime	Time functions for dates
iomanip	Formatting output
limits	Numeric limits for validation
map	Exchange rates storage

Table 2: Header Files Used

Method	Description	Parameters
deposit	Adds amount to balance	double amount, Currency cur
withdraw	Subtracts if sufficient	double amount, Currency cur
transfer	Transfers to another account	Account &recipient, double amount, Curren
applyForLoan	Adds loan, credits principal	double principal, Currency cur, double rate,

Table 3: Key Account Methods

5.3 Key Functions

5.4 Code Features

- **OOP**: Encapsulation, abstraction.
- **Data Structures**: Vector, map.
- **Error Handling**: Input validation.
- **Modularity**: Separated concerns.

Sample Code:

```
void applyForLoan(double principal, Currency currency, double interestRat
    double principalInPKR = convertCurrency(principal, currency, CUR_PKR)
    loanRecords.push_back(Loan(principalInPKR, interestRate, termMonths))
    accountBalanceInPKR += principalInPKR;
    transactionHistory.push_back({currentDateTime(), TRANSACTION_DEPOSIT,
}
```

6 How to Run the Program

1. **Compile**: `g++ banking_system.cpp -o banking_system`
2. **Run**: `./banking_system`
3. Follow menu prompts for operations.

Note: Requires C++11 or later.

7 Limitations and Future Improvements

Limitations:

- In-memory data; no persistence.
- Simple interest; no compounding.
- Fixed exchange rates.
- Basic security; no encryption.
- No GUI or concurrency.

Future Improvements:

- Add file I/O for persistence.
- Implement compound interest.
- Integrate real-time currency APIs.
- Enhance security with hashing.
- Develop graphical interface (e.g., Qt).

8 Conclusion

The **Banking Management System** demonstrates C++ proficiency in a practical banking context. It serves as a foundation for advanced features and showcases our team's collaborative effort.

> *"In the world of programming, every line of code is a step towards innovation."* – Anonymous